### Leap Year Definition

A leap year in the Gregorian calendar has an extra day, February 29th. To decide if a year is a leap year, we use these rules. First, if a year can be divided evenly by 4, it is usually a leap year. But if the year is divisible by 100, it is not a leap year, except when it is also divisible by 400. For example, 2024 can be divided by 4, so it's a leap year. 1900 can be divided by 4 and 100, but not by 400, so it's not a leap year. While 2000 can be divided by 4, 100, and 400, so it is a leap year.

**Probability Distribution of Leap Years**

1. **In a 4 - year cycle**:
   - In every 4 - year cycle, there is one leap year. So if we randomly pick a year from a 4 - year cycle, the probability that it is a leap year is $\frac{1}{4}$ because out of 4 years, 1 is a leap year.
2. **In a 100 - year period**:
   - In a 100 - year period, there are 25 years divisible by 4. But since century years (ending with 00) that are not divisible by 400 are not leap years, and there are 3 such century years in a 100 - year period, the number of leap years is 24. So the probability of a randomly chosen year in a 100 - year period being a leap year is $\frac{24}{100}$ as there are 24 leap years out of 100 years.
3. **In a 400 - year period**:
   - In a 400 - year period, there are 100 years divisible by 4. Among the 4 century years, 3 are not divisible by 400. So the number of leap years is 97. The probability of a randomly chosen year in a 400 - year period being a leap year is $\frac{97}{400}$ because there are 97 leap years out of 400 years.

Let's consider the probability of a given day being February 29th using the concept of conditional probability.

We know that in a 400-year period, there are 97 leap years (years with February 29th). The total number of days in 400 years is $400 \times 365 + 97$ (since there are 97 extra days in the leap years).

Let $A$ be the event that a day is February 29th and $B$ be the event that a year is a leap year.

The probability of a year being a leap year, $P(B)$, is $\frac{97}{400}$.

Given that a year is a leap year, the probability that a particular day within that year is February 29th, $P(A|B)$, is $\frac{1}{366}$.

We want to find $P(A)$, the probability that a randomly chosen day is February 29th.

By the law of total probability, $P(A) = P(A|B)P(B)$.

Substituting the values we have:

$$P(A) = \frac{1}{366} \times \frac{97}{400} = \frac{97}{366 \times 400} = \frac{97}{146400} \approx 0.000663$$

So the probability that a randomly chosen day is February 29th is approximately $0.000663$.

Consider a general situation where we assume there are 365 days in a year (ignoring leap years for now) and there are $n$ people. We need to find the probability that at least two of these $n$ people have the same birthday. A good way to do this is to first find the probability that all $n$ people have different birthdays and then subtract this probability from 1 to get the probability that at least two people have the same birthday.

**Calculation of the Probability that All Birthdays are Different**

- For the first person, his or her birthday can be any of the 365 days, so the probability is 1 (or written as $\frac{365}{365}$).
- When considering the second person, to ensure that his or her birthday is different from the first person's, there are only 364 choices for his or her birthday. So the probability that the second person's birthday is different from the first person's is $\frac{364}{365}$.
- For the third person, to ensure that his or her birthday is different from the first two people's, there are 363 choices for his or her birthday. So the probability that the third person's birthday is different from the first two people's is $\frac{363}{365}$.
- By analogy, for the $n$th person, the probability that his or her birthday is different from the previous $n-1$ people's is $\frac{365-(n-1)}{365}$.

Then the probability $P(\text{All birthdays are different})$ that all $n$ people have different birthdays is the product of these probabilities, that is $P(\text{All birthdays are different}) = \frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \cdots \times \frac{365-n+1}{365}$. This formula can also be written as $P(\text{All birthdays are different}) = \frac{365!}{(365-n)! \times 365^n}$.

**Calculation of the Probability that At Least Two People Have the Same Birthday**

- Let $P(\text{At least two people have the same birthday})$ be the probability we want to find.
- Then

  $P(\text{At least two people have the same birthday}) = 1 - P(\text{All birthdays are different}).$

**Question 1**: Find the second smallest number without using sorting.

**Answer**: One way to do this is to first find the minimum number in the array by iterating through it once and keeping track of the minimum value. Then, initialize a variable to store the second smallest number with a very large value (say, `Integer.MAX_VALUE` in Java). Iterate through the array again, and if the current number is not equal to the minimum number we found earlier and is smaller than the current second smallest number, update the second smallest number. Here is the Java code example:

```
def find_second_smallest(arr):
    # 找到最小的数
    min_num = float('inf')
    for num in arr:
        if num < min_num:
            min_num = num

    # 找到第二小的数
    second_smallest = float('inf')
    for num in arr:
        if num!= min_num and num < second_smallest:
            second_smallest = num
    return second_smallest
```

**Question 2**: Given an irrational number and a rational number, prove that there must be an irrational number between them.

**Answer**: Let $a$ be an irrational number and $b$ be a rational number such that $a < b$. Consider the number $c = \frac{a+b}{2}$.

We will prove that $c$ is irrational. Suppose, for the sake of contradiction, that $c$ is rational. Then $c = \frac{m}{n}$ for some integers $m$ and $n$, $n \neq 0$

We have $c = \frac{a+b}{2}$, so $a + b = 2c$

If $c$ is rational, then $2c$ is also rational. Since $b$ is rational, if $a + b$ is rational, then $a = (a + b) - b$ must be rational (because the difference of two rational numbers is rational), which contradicts the fact that $a$ is irrational.

So $c = \frac{a+b}{2}$ is irrational and $a < c < b$

1. 首先，考虑数$c = \frac{a+b}{2}$。
2. 假设$c$是有理数，那么$c = \frac{m}{n}$，其中$m$和$n$是整数且$n \neq 0$。
3. 因为$c = \frac{a+b}{2}$，所以$a + b = 2c$。
4. 如果$c$是有理数，那么$2c$也是有理数。由于$b$是有理数，如果$a + b$是有理数，那么$a = (a + b) - b$也必须是有理数（因为两个有理数的差是有理数），这与$a$是无理数的事实相矛盾。
5. 所以$c = \frac{a+b}{2}$是无理数，并且$a < c < b$。

这个证明通过反证法，假设$c$是有理数，然后推出矛盾，从而证明了$c$是无理数，也就说明了在无理数$a$和有理数$b$之间存在无理数$c$。

**Question 3**: Write pseudocode for sharp sort (which is a variant of bubble sort).

```
procedure sharpSort(array A)
    n = length(A)
    repeat
        swapped = false
        for i = 1 to n - 1
            if A[i] > A[i + 1]
                swap A[i] and A[i + 1]
                swapped = true
            end if
        end for
        for i = n - 1 downto 1
            if A[i] < A[i - 1]
                swap A[i] and A[i - 1]
                swapped = true
            end if
        end for
    until not swapped
end procedure
```

1. **初始化**
   - `n = length(A)`：获取数组 `A` 的长度，并将其存储在变量 `n` 中。
2. **主循环**
   - `repeat`：开始一个循环，该循环会一直执行，直到在一次完整的遍历中没有发生任何交换操作（由 `swapped` 变量控制）。
   - `swapped = false`：在每次循环开始时，将 `swapped` 变量初始化为 `false`，用于标记在本次遍历中是否发生了元素交换。
3. **正向遍历（从前往后）**
   - `for i = 1 to n - 1`：从数组的第一个元素（索引为 1）开始，到倒数第二个元素（索引为 `n - 1`）结束，进行遍历。
   - `if A[i] > A[i + 1]`：对于每一对相邻的元素 `A[i]` 和 `A[i + 1]`，如果 `A[i]` 大于 `A[i + 1]`，则进行交换操作。
     - `swap A[i] and A[i + 1]`：交换 `A[i]` 和 `A[i + 1]` 的位置，使较大的元素向后移动。
     - `swapped = true`：将 `swapped` 变量设置为 `true`，表示在本次遍历中发生了交换操作。
4. **反向遍历（从后往前）**
   - `for i = n - 1 downto 1`：从数组的倒数第二个元素（索引为 `n - 1`）开始，到第一个元素（索引为 1）结束，进行反向遍历。
   - `if A[i] < A[i - 1]`：对于每一对相邻的元素 `A[i]` 和 `A[i - 1]`，如果 `A[i]` 小于 `A[i - 1]`，则进行交换操作。
     - `swap A[i] and A[i - 1]`：交换 `A[i]` 和 `A[i - 1]` 的位置，使较小的元素向前移动。
     - `swapped = true`：将 `swapped` 变量设置为 `true`，表示在本次遍历中发生了交换操作。
5. **循环结束条件**
   - `until not swapped`：只要在一次完整的正向和反向遍历中发生了交换操作（`swapped` 为 `true`），就会继续执行循环。当在一次完整的遍历中没有发生任何交换操作时（`swapped` 为 `false`），表示数组已经有序，循环结束。

**Question 4**: Find the monotonic intervals and then calculate an integral for finding an area.

**Answer**: To find the monotonic intervals of a function $y = f(x)$, we first find the derivative $f'(x)$.

The function $y = f(x)$ is increasing when $f'(x) > 0$ and decreasing when $f'(x) < 0$

To calculate the area between the curve $y = f(x)$, the $x$-axis, and the lines $x = a$ and $x = b$, we calculate the definite integral $\int_a^b f(x)dx$

For example, if $f(x) = x^2$ on the interval $[0, 2]$

The derivative $f'(x) = 2x$, $f'(x) > 0$ when $x > 0$ so the function is increasing on $(0, \infty)$

The area under the curve from $x = 0$ to $x = 2$ is $\int_0^2 x^2 dx = \left[\frac{x^3}{3}\right]_0^2 = \frac{8}{3}$

## Problem 1: Find elements larger than their four surrounding elements in a 2D array

**Question**: Given a 2D array, write a program to find all the elements that are larger than their four surrounding elements (up, down, left, and right, assuming valid indices within the array boundaries). For example, if we have a 2D array `arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`, we need to check each element in the array to see if it meets the criteria.

```python
def find_larger_than_surrounding(arr):
    rows = len(arr)
    cols = len(arr[0])
    result = []
    for i in range(rows):
        for j in range(cols):
            is_larger = True
            if i > 0 and arr[i][j] <= arr[i - 1][j]:
                is_larger = False
            if i < rows - 1 and arr[i][j] <= arr[i + 1][j]:
                is_larger = False
            if j > 0 and arr[i][j] <= arr[i][j - 1]:
                is_larger = False
            if j < cols - 1 and arr[i][j] <= arr[i][j + 1]:
                is_larger = False
            if is_larger:
                result.append(arr[i][j])
    return result


arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(find_larger_than_surrounding(arr))
```

**题目描述：**

有一个背包，其容量为$C$，现有$n$个物品，每个物品有重量$w_i$和价值$v_i$。贪心算法的一种策略是优先选择单位重量价值（$v_i/w_i$）最大的物品放入背包，直到背包不能再放入物品为止。请给出一个反例，说明这种贪心策略不能得到最优解。

**解释：**

- 例如，背包容量$C = 10$，有三个物品：
  - 物品$A$：重量$w_1 = 3$，价值$v_1 = 4$，单位重量价值为$4/3 \approx 1.33$。
  - 物品$B$：重量$w_2 = 4$，价值$v_2 = 5$，单位重量价值为$5/4 = 1.25$。
  - 物品$C$：重量$w_3 = 5$，价值$v_3 = 7$，单位重量价值为$7/5 = 1.4$。

按照贪心算法，优先选择单位重量价值最大的物品$C$放入背包，此时背包剩余容量为$10 - 5 = 5$，无法再放入其他物品，得到的总价值为$7$。

然而，如果我们不采用贪心策略，而是选择物品$A$和物品$B$放入背包，此时总重量为$3 + 4 = 7$，小于背包容量$10$，总价值为$4 + 5 = 9$，$9 > 7$，说明贪心算法得到的不是最优解。

**Problem Description:**

There is a knapsack with capacity $C$, and there are $n$ items. Each item has a weight $w_i$ and a value $v_i$. One strategy of the greedy algorithm is to preferentially select the item with the largest value per unit weight ($v_i/w_i$) to put into the knapsack until the knapsack can no longer hold any items. Please give a counterexample to show that this greedy strategy may not obtain the optimal solution.

**Explanation:**

- For example, the knapsack capacity $C = 10$, and there are three items:
  - Item $A$: weight $w_1 = 3$, value $v_1 = 4$, and the value per unit weight is $4/3 \approx 1.33$.
  - Item $B$: weight $w_2 = 4$, value $v_2 = 5$, and the value per unit weight is $5/4 = 1.25$.
  - Item $C$: weight $w_3 = 5$, value $v_3 = 7$, and the value per unit weight is $7/5 = 1.4$.

According to the greedy algorithm, we first select item $C$ with the largest value per unit weight to put into the knapsack. At this time, the remaining capacity of the knapsack is $10 - 5 = 5$, and no other items can be put in, and the total value obtained is $7$.
However, if we do not use the greedy strategy but choose to put item $A$ and item $B$ into the knapsack, the total weight is $3 + 4 = 7$, which is less than the knapsack capacity $10$, and the total value is $4 + 5 = 9$. Since $9 > 7$, it shows that the solution obtained by the greedy algorithm is not the optimal solution.

**题目：**
给定一个字符串 "hello world"，使用以下加密规则对其进行加密：
对于字符串中的小写字母，将其 ASCII 码值先加上当前字母在字符串中的位置（从 1 开始计数），然后对 26 取模，得到的结果再转换回小写字母。

•对于字符串中的空格，保持不变。
例如，对于字母 'h'，它是字符串中的第 1 个字母，'h' 的 ASCII 码值为 104，加密后为 (104 + 1) % 26 对应的小写字母。

```
def advanced_caesar_encrypt(s):
    encrypted = ""
    for i, char in enumerate(s, 1):
        if char.islower():
            # 计算加密后的ASCII码值
            encrypted_ascii = (ord(char) - ord('a') + i) % 26 + ord('a')
            # 转换回字符并添加到加密后的字符串
            encrypted += chr(encrypted_ascii)
        else:
            # 非小写字母（这里是空格）直接添加
            encrypted += char
    return encrypted
```